



CSCE 633-700: Machine Learning

Linear regression

Overview

Linear Regression

- Example
- Representation, Evaluation
- Optimization: Closed form solution via Ordinary Least Squares
- Optimization: Numerical solution via Gradient Descent
 - General gradient descent
 - Gradient descent for linear regression (batch, stochastic, mini-batch)
- Non-linear basis function for regression & Overfitting

[Parts of these slides have been adapted from K. Murphy (Machine Learning: A probabilistic perspective), Dr. Andrew Ng's Machine Learning course at Coursera, and CSCI567 Machine Learning (USC, Drs. Sha & Liu)]

Gradient Descent in Linear Regression

We can now derive the algorithm outline for minimizing the residual square sum (RSS) error of linear regression with gradient descent

- The residual sum of squares is the cost function:

$$\begin{aligned}J(\mathbf{w}) &= RSS(\mathbf{w}) = (\mathbf{y} - \mathbf{X}\mathbf{w})^T(\mathbf{y} - \mathbf{X}\mathbf{w}) \\&= \mathbf{y}^T\mathbf{y} - 2(\mathbf{X}\mathbf{w})^T\mathbf{y} + (\mathbf{X}\mathbf{w})^T(\mathbf{X}\mathbf{w}) \\&= \mathbf{y}^T\mathbf{y} - 2\mathbf{w}^T(\mathbf{X}^T\mathbf{y}) + \mathbf{w}^T(\mathbf{X}^T\mathbf{X})\mathbf{w}\end{aligned}$$

$$\mathbf{w} := \mathbf{w} - \alpha(k) \cdot \nabla J(\mathbf{w})$$

- Gradient Descent optimization expression:

$$\nabla J(\mathbf{w}) = \frac{\partial RSS(\mathbf{w})}{\partial \mathbf{w}} = -2\mathbf{X}^T\mathbf{y} + 2\mathbf{X}^T\mathbf{X}\mathbf{w}$$

Gradient Descent in Linear Regression

Question: Derive the algorithm outline for minimizing the residual square sum (RSS) error of linear regression with gradient descent

(Batch) Gradient Descent for Linear Regression

- 1 Initialize $\mathbf{w}$, ϵ , $\alpha(\cdot)$, $k := 0$
- 2 While $\|\nabla RSS(\mathbf{w})\|_2 > \epsilon$
 - 2a $k := k + 1$
 - 2b $\mathbf{w} := \mathbf{w} - \alpha(k) \cdot (\mathbf{X}^T \mathbf{X} \mathbf{w} - \mathbf{X}^T \mathbf{y})$

Gradient Descent in Linear Regression

Stochastic Gradient Descent for Linear Regression

Update weights using one sample at a time

- 1 Initialize $\mathbf{w}$, ϵ , $\alpha(\cdot)$, $k := 0$
- 2 Loop until convergence
 - 2a $k := k + 1$
 - 2b Randomly choose a sample $(\mathbf{x}_i, y_i)$
 - 2c Compute its contribution to the gradient $\mathbf{g}_i = (\mathbf{x}_i^T \mathbf{w} - y_i) \cdot \mathbf{x}_i$
 - 2d Update the weights $\mathbf{w} := \mathbf{w} - \alpha(k) \cdot \mathbf{g}_i$

Gradient Descent in Linear Regression

Mini-Batch Gradient Descent for Linear Regression

Update weights using subset of samples at a time

- 1 Initialize $\mathbf{w}$, ϵ , $\alpha(\cdot)$, $k := 0$
- 2 Loop until convergence
 - 2a $k := k + 1$
 - 2b Randomly choose a subset of samples $S = \{(\mathbf{x}_i, y_i), \dots, (\mathbf{x}_{i+M}, y_{i+M})\}$
 - 2c Form the mini-batch data matrix $\mathbf{X}_S = \begin{bmatrix} \mathbf{x}_i^T \\ \vdots \\ \mathbf{x}_{i+M}^T \end{bmatrix}$
 - 2d Update the weights $\mathbf{w} := \mathbf{w} - \alpha(k) \cdot (\mathbf{X}_S^T \mathbf{X}_S \mathbf{w} - \mathbf{X}_S^T \mathbf{y})$
- Good compromise between batch and stochastic gradient descent
- Common mini-batch sizes range between $M=50$ -250 samples

Gradient Descent in Linear Regression

- **Batch** gradient descent computes exact gradient
- **Stochastic** gradient descent
 - Computes approximate gradient using one sample per iteration
 - Its expectation equals the true gradient
- **Mini-batch** gradient descent
 - Computes gradient based on subset of samples
- For large-scale problems stochastic or mini-batch descent often work well